

CSS: responsive design e media query

Un sito moderno deve funzionare bene su schermi molto diversi: da un telefono a un monitor largo. Il responsive design è la tecnica che lo rende possibile.

■ L'approccio mobile-first

Nell'approccio **mobile-first** si scrive prima il CSS pensando allo schermo più piccolo (il caso più vincolato), poi si aggiungono regole per schermi via via più larghi con le **media query**. È l'approccio consigliato, perché costringe a partire dall'essenziale invece di "nascondere" elementi superflui su schermi piccoli.

```
/* stile di base: pensato per mobile */
.scheda {
  display: block;
  width: 100%;
}

/* da qui in su, schermo più largo di un tablet */
@media (min-width: 768px) {
  .scheda {
    display: flex;
    width: 48%;
  }
}
```

■ Il viewport e i breakpoint comuni

Perché le media query funzionino correttamente sui dispositivi mobili, ogni pagina deve avere nell'<head> il tag <meta name="viewport">, che dice al browser di usare la larghezza reale dello schermo invece di simulare un desktop. I breakpoint più comuni (non regole fisse, solo convenzioni) sono intorno a 480px (telefoni), 768px (tablet) e 1024px (desktop).

```
<meta name="viewport" content="width=device-width, initial-scale=1" />
```

- @media (min-width: 480px) — da qui in su, smartphone in orizzontale e tablet piccoli.
- @media (min-width: 768px) — tablet e schermi medi.
- @media (min-width: 1024px) — desktop.

■ Caso d'uso: dal menu orizzontale al menu hamburger

Un problema molto pratico del responsive design: un menu con 6 voci sta comodamente in orizzontale su un monitor, ma non su uno smartphone. La soluzione standard è nascondere il menu orizzontale sotto una certa larghezza e mostrare al suo posto un pulsante "hamburger" (≡) che, cliccato, apre un pannello con gli stessi link, di solito impilati verticalmente.

```
.menu-desktop { display: flex; }  
.pulsante-hamburger { display: none; }  
  
@media (max-width: 767px) {  
  .menu-desktop { display: none; }  
  .pulsante-hamburger { display: block; }  
}
```

Il CSS da solo può gestire il cambio di aspetto tra le due modalità, ma l'apertura e chiusura effettiva del pannello al clic richiede una piccola porzione di JavaScript (che aggiunge o toglie una classe, ad esempio `.aperto`, sul pannello del menu) — un primo esempio di come CSS e JavaScript collaborano nella stessa interfaccia, ciascuno con il proprio compito: JavaScript decide **quando** qualcosa cambia, CSS decide **come** appare quel cambiamento.

ESERCIZI DI RIEPILOGO

1. Prendi una pagina con una griglia a 3 colonne e aggiungi una media query che la trasformi in una singola colonna sotto i 600px.
2. Aggiungi il tag `viewport` corretto a una pagina che non lo ha, e verifica la differenza su uno schermo stretto.
3. Descrivi a parole cosa significa “mobile-first” e perché, secondo te, è preferibile scrivere prima lo stile per lo schermo piccolo.